

HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA ĐIỆN TỬ - VI MẠCH



BÁO CÁO MÔN HỌC
HỆ THỐNG NHÚNG THỜI GIAN THỰC
THIẾT KẾ ĐỒNG HỒ THỜI GIAN THỰC CÓ LED
NHÁY THEO NHẠC SỬ DỤNG HỆ ĐIỀU HÀNH
FREERTOS

Giảng viên hướng dẫn: THS Lê Thị Hồng Vân

Sinh viên thực hiện: Hà Văn Đạt – DT060209

Dương Hải Đăng – DT060206

Nguyễn Mạnh Lân – DT060231

Hà Nội – 2026

LỜI NÓI ĐẦU

Cùng với sự phát triển không ngừng của xã hội hiện đại, nhu cầu giải trí và nâng cao đời sống tinh thần ngày càng được chú trọng. Trong đó, âm nhạc đóng vai trò như một liệu pháp tinh thần thiết yếu giúp giải tỏa căng thẳng và khơi nguồn sáng tạo. Để tối ưu hóa trải nghiệm đa giác quan của người dùng, sự kết hợp giữa âm thanh và công nghệ ánh sáng đang trở thành một xu hướng tất yếu trong các thiết bị công nghệ hiện đại.

Sự ra đời và tiến bộ của công nghệ LED đã tạo ra một bước đột phá lớn, định hình lại diện mạo của ngành công nghiệp giải trí. Đặc biệt, sự giao thoa đồng bộ giữa nhịp điệu âm thanh và hiệu ứng ánh sáng luôn tạo ra sức hút mạnh mẽ.

Xuất phát từ thực tiễn đó, cùng với mong muốn vận dụng các kiến thức chuyên môn đã tích lũy từ các học phần liên quan đến hệ thống nhúng, nhóm chúng em đã quyết định nghiên cứu và thực hiện đề tài: Thiết kế đồng bộ thời gian thực có Led nhấp nháy theo nhạc sử dụng hệ điều hành. Điểm nhấn kỹ thuật của đề án này là việc sử dụng vi điều khiển STM32F4 thay vì các dòng vi điều khiển 8-bit cơ bản. Với kiến trúc lõi ARM Cortex-M4 mạnh mẽ cùng tập lệnh hỗ trợ xử lý tín hiệu số (DSP), hệ thống có khả năng thu thập và phân tích phổ âm thanh theo thời gian thực (Real-time) với độ chính xác cao và độ trễ cực thấp. Từ đó, đề án mang đến những hiệu ứng nhấp LED mượt mà, phức tạp và bám sát nhịp điệu âm nhạc, giúp người dùng có những phút giây thư giãn trọn vẹn nhất.

Để hoàn thành được đề án và cuốn báo cáo này, trước hết, nhóm chúng em xin gửi lời cảm ơn sâu sắc tới Cô Lê Thị Hồng Vân – Giảng viên khoa Công nghệ thông tin, Học viện Kỹ thuật Mật mã. Cô đã luôn tận tình định hướng, chỉ bảo và hỗ trợ giải quyết các vấn đề kỹ thuật trong suốt thời gian chúng em nghiên cứu, thực hiện đề tài.

Mặc dù đã có nhiều cố gắng, song do giới hạn về mặt thời gian và kinh nghiệm thực tiễn, đề án chắc chắn không tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được sự góp ý của Cô để sản phẩm được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

LỜI NÓI ĐẦU	I
MỤC LỤC	II
DANH MỤC TỪ VIẾT TẮT	IV
DANH MỤC HÌNH ẢNH.....	V
DANH MỤC BẢNG BIỂU	VI
CHƯƠNG 1 : TỔNG QUAN	1
1.1. Đặt vấn đề	1
1.2. Lý do chọn đề tài.....	1
1.3. Mục đích nghiên cứu đề tài.....	2
CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT	3
2.1. Tổng quan về hệ điều hành FREERTOS	3
2.1.1. Lịch sử ra đời và phát triển	3
2.1.2. Định nghĩa về FreeRTOS.....	3
2.1.3. Thành phần chính của FreeRTOS.....	4
2.1.4. Ứng dụng của hệ điều hành FreeRTOS	4
2.2. Hệ thống đồng hồ thời gian thực (RTC)	5
2.2.1. Khái niệm RTC	5
2.2.2. Nguyên lý hoạt động của RTC.....	5
2.2.3. Phân loại RTC	6
2.2.4. Vai trò của RTC trong đề tài.....	6
2.3. Xử lý tín hiệu âm thanh và điều khiển LED RGB theo nhạc	7
2.3.1. Khái niệm tín hiệu âm thanh số	7
2.3.2. Nguyên lý hoạt động của cảm biến âm thanh IMNP441	7
2.3.3. Phương pháp xử lý tín hiệu âm thanh	8
2.3.4. Nguyên lý điều khiển LED WS2812	8
2.3.5. Điều khiển LED RGB theo nhạc	8
2.3.6. Kết hợp với hệ điều hành FreeRTOS.....	9
CHƯƠNG 3 : THIẾT KẾ VÀ THI CÔNG	10

3.1. Sơ đồ khối của hệ thống.....	10
3.2. Thi công phần cứng.....	11
3.2.1. Các linh kiện sử dụng.....	11
3.2.2. Sơ đồ kết nối:	18
3.2. Thi công phần mềm.....	19
3.2.1. Cấu hình trên STM32 CubeMX.....	19
3.2.2. Triển khai chương trình	21
CHƯƠNG 4 : KẾT QUẢ VÀ ĐÁNH GIÁ	26
4.1. Sản phẩm thực tế.....	26
4.2 Kiểm tra và đánh giá	27
4.2.1. Kết quả thực hiện các chức năng chính	27
4.2.2. Đánh giá hiệu năng hệ thống	27
KẾT LUẬN	29
1. Ưu điểm.....	29
2. Nhược điểm.....	29
3. Phương hướng phát triển.....	30
TÀI LIỆU THAM KHẢO	31

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
ADC	Bộ chuyển đổi tương tự sang số
AWS	Dịch vụ điện toán đám mây
CPU	Bộ xử lý trung tâm
DMA	Truy cập bộ nhớ trực tiếp
DSP	Xử lý tín hiệu số
EXTI	Ngắt ngoài
GPIO	Cổng vào/ra đa năng
HAL	Thư viện phát triển phần mềm cho STM
HCLK	Xung nhịp hệ thống
IC	Mạch tích hợp
MEMS	Hệ thống vi cơ điện tử
OS	Hệ điều hành
PWM	Điều chế độ rộng xung
RGB	Hệ màu đỏ - xanh lá - xanh dương
RTC	Đồng hồ thời gian thực
SNR	Tỷ lệ tín hiệu trên nhiễu
SRAM	Bộ nhớ truy cập ngẫu nhiên tĩnh

DANH MỤC HÌNH ẢNH

Hình 1.1. Led nháy theo nhạc phổ biến trên thị trường.....	1
Hình 2.1 : Real Time Clock Hardware Features.....	6
Hình 3.1 : Sơ đồ khối	10
Hình 3.2 : STM32F411CEU6	11
Hình 3.3 : Module DS1307	13
Hình 3.4 : IMNP441.....	14
Hình 3.5: DFPLAYER.....	15
Hình 3.6 : OLED SSD 1306.....	16
Hình 3.7 : LED WS2812.....	17
Hình 3.8 : Nút nhấn switch	18
Hình 3.9 : Sơ đồ kết nối của hệ thống.....	18
Hình 3.10 :Cấu hình thạch anh và Debug	19
Hình 3.11 : Cấu hình GPIO.....	19
Hình 3.12 : Cấu hình I2S	20
Hình 3.13 : Cấu hình I2C	21
Hình 3.14 : Cấu hình FreeRTOS.....	21
Hình 3.15 : Lưu đồ giải thuật tổng thể của hệ thống	22
Hình 3.16 : Thuật toán mô tả chương trình task I2S_ledTASK	23
Hình 3.17 : Thuật toán mô tả chương trình task I2C_TimeTask.....	24
Hình 3.18 : Thuật toán mô tả chương trình task I2C_Time_ClockTask	25
Hình : Sản phẩm thực tế.....	26
Hình 4.2: Sản phẩm thực tế 2.....	26

DANH MỤC BẢNG BIỂU

Bảng 2.1: Các thành phần chính của FreeRTOS	4
Bảng 2.2: Ứng dụng của FreeRTOS	5
Bảng 3.1 : Thông số kỹ thuật của STM32F411CEU6.....	12
Bảng 3.2 : Thông số của DS1307:	14
Bảng 3.3 : Thông số kỹ thuật IMNP441	15
Bảng 3.4: Thông số của OLED SSD1306	16
Bảng 3.5: Thông số của LED WS2812.....	17
Bảng 4.1: Kết quả thử nghiệm các tính năng của hệ thống	27

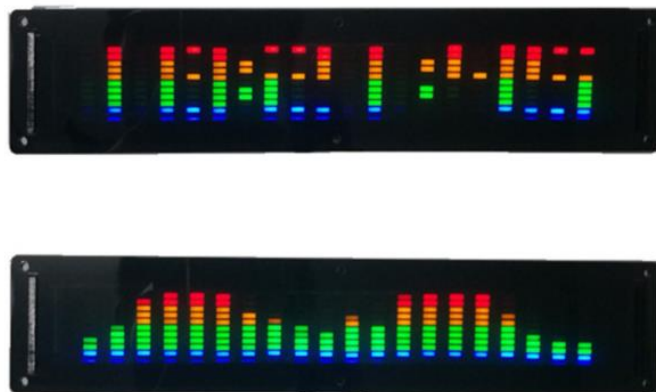
CHƯƠNG 1 : TỔNG QUAN

1.1. Đặt vấn đề

Cùng với sự tiến bộ không ngừng của khoa học kỹ thuật và sự bùng nổ của thiết bị thông minh, các sản phẩm điện tử gia dụng ngày nay không chỉ dừng lại ở việc đáp ứng các công năng cơ bản mà còn phải thỏa mãn nhu cầu thẩm mỹ và nâng cao trải nghiệm người dùng. Đối với thiết bị đo lường thời gian, sự chuyển dịch từ đồng hồ cơ học sang đồng hồ kỹ thuật số đã giải quyết triệt để các bài toán về độ chính xác, chi phí và khả năng hiển thị. Tuy nhiên, phần lớn các thiết bị trên thị trường hiện nay vẫn giới hạn ở việc cung cấp thông số thuần túy, thiếu đi sự tương tác sinh động. Từ thực tiễn đó, việc phát triển một hệ thống thiết bị kết hợp giữa tính năng đếm thời gian thực (Real-time) và công nghệ xử lý tín hiệu âm thanh để điều khiển hiệu ứng ánh sáng (Audio-reactive LED) là một bài toán kỹ thuật hấp dẫn và mang tính ứng dụng cao.

1.2. Lý do chọn đề tài

Hiện nay, trên thị trường chủ yếu cung cấp các loại đồng hồ cơ bản hoặc các dải LED nháy theo nhạc với bộ vi xử lý tín hiệu rất thô sơ (thường chỉ nháy đơn giản theo cường độ âm thanh - Volume). Để khắc phục hạn chế này và mang lại một trải nghiệm mượt mà, chuyên nghiệp hơn, nhóm quyết định thực hiện đề tài: “Thiết kế hệ thống đồng hồ thời gian thực kết hợp LED hiệu ứng âm thanh”.



Hình 1.1. Led nháy theo nhạc phổ biến trên thị trường

Lý do cốt lõi dẫn đến việc lựa chọn đề tài này bao gồm:

Sức mạnh từ nền tảng phần cứng: Thay vì sử dụng các dòng vi điều khiển 8-bit truyền thống, hệ thống ứng dụng vi điều khiển STM32F4 với kiến trúc ARM Cortex-M4 mạnh mẽ. Điều này cho phép thực hiện các thuật toán xử lý tín hiệu số (DSP) và lấy mẫu tín hiệu (ADC) tốc độ cao để phân tích phổ âm thanh phức tạp một cách chính xác.

Tính ưu việt của hệ điều hành thời gian thực: Việc ứng dụng FreeRTOS giúp quản lý đa nhiệm hiệu quả, đảm bảo tính đồng bộ tuyệt đối giữa việc cập nhật thời gian từ RTC, xử lý dữ liệu âm thanh và điều khiển LED mà không gây xung đột hệ thống.

Trải nghiệm người dùng: Sự kết hợp giữa khả năng hiển thị thời gian ổn định và hiệu ứng ánh sáng mượt mà, bám sát nhịp điệu âm nhạc sẽ tạo ra một thiết bị vừa mang tính hữu dụng vừa mang tính nghệ thuật cao.

1.3. Mục đích nghiên cứu đề tài

Đề tài được thực hiện nhằm hướng tới các mục tiêu cụ thể sau:

Về mặt lý thuyết: Nghiên cứu sâu về kiến trúc phần cứng của dòng vi điều khiển STM32F4, kỹ thuật lập trình nhúng, phương pháp giao tiếp ngoại vi (I2C, SPI...) và đặc biệt là kỹ thuật thu thập, xử lý tín hiệu số (DSP).

Về mặt thực tiễn: Thiết kế và thi công hoàn thiện một thiết bị đồng hồ đa năng. Hệ thống phải đảm bảo hoạt động ổn định, cập nhật thời gian chính xác, giao diện người dùng trực quan và tính năng ánh sáng phản hồi âm thanh hoạt động mượt mà với độ trễ thấp (Real-time).

CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về hệ điều hành FREERTOS

FreeRTOS là một hệ điều hành thời gian thực (Real-Time Operating System RTOS) mã nguồn mở, được thiết kế chuyên biệt cho các hệ thống nhúng có tài nguyên hạn chế như vi điều khiển (microcontroller).

2.1.1. Lịch sử ra đời và phát triển

FreeRTOS được phát triển lần đầu vào năm 2003 bởi Richard Barry. Ban đầu, mục tiêu của ông là tạo ra một RTOS nhỏ gọn, dễ sử dụng và miễn phí cho cộng đồng phát triển hệ thống nhúng.

Đến năm 2017, FreeRTOS được Amazon mua lại và phát triển thành Amazon FreeRTOS, mở rộng thêm các tính năng hỗ trợ Internet of Things (IoT), bảo mật và kết nối cloud (AWS).

2.1.2. Định nghĩa về FreeRTOS

Hệ điều hành (Operating System – OS) là phần mềm nền tảng đóng vai trò trung gian giữa người dùng và phần cứng của thiết bị. Nó cho phép các ứng dụng khác hoạt động một cách hiệu quả trên hệ thống. Thông qua hệ điều hành, người dùng có thể quản lý, điều khiển tài nguyên phần cứng như CPU, bộ nhớ, thiết bị ngoại vi, cũng như tổ chức và vận hành các chương trình phần mềm một cách tối ưu.

Theo tài liệu chính thức từ FreeRTOS (trên trang freertos.org):” FreeRTOS là một hệ điều hành thời gian thực nhẹ, được thiết kế để chạy trên các hệ thống nhúng với yêu cầu thời gian đáp ứng chính xác và hiệu suất cao.”

FreeRTOS là một hệ điều hành thời gian thực được thiết kế với tiêu chí nhỏ gọn, hiệu quả và dễ triển khai trên các hệ thống nhúng. Nhờ kích thước kernel rất nhỏ, FreeRTOS có thể hoạt động tốt trên các vi điều khiển có tài nguyên hạn chế như RAM và Flash thấp. Bên cạnh đó, hệ điều hành này hỗ trợ cơ chế đa nhiệm

(multitasking), cho phép nhiều tác vụ chạy song song mà vẫn đảm bảo tính chính xác về thời gian.

Ngoài ra, FreeRTOS còn có tính linh hoạt cao, dễ dàng port sang nhiều nền tảng phần cứng khác nhau như STM32, ESP32, AVR,... Đồng thời, do là mã nguồn mở và được cộng đồng hỗ trợ rộng rãi, việc học tập và phát triển ứng dụng với FreeRTOS trở nên thuận tiện hơn rất nhiều.

2.1.3. Thành phần chính của FreeRTOS

Các thành phần chính của FreeRTOS được thể hiện qua bảng sau:

Bảng 2.1: Các thành phần chính của FreeRTOS

Thành phần	Chức năng
Task	Là các chương trình con hoạt động độc lập, có thể chạy song song. Mỗi task có mức ưu tiên riêng.
Scheduler	Bộ lập lịch có nhiệm vụ quyết định task nào được thực thi dựa trên độ ưu tiên và trạng thái.
Queue	Cơ chế truyền dữ liệu giữa các task một cách an toàn, tránh xung đột dữ liệu.
Semaphore	Dùng để đồng bộ hóa giữa các task hoặc giữa task với ngắt.
Mutex	Một dạng semaphore đặc biệt dùng để bảo vệ tài nguyên dùng chung.
Timer	Hỗ trợ thực hiện các tác vụ theo thời gian định trước mà không cần delay thủ công.
Memory Management	Quản lý cấp phát và giải phóng bộ nhớ với nhiều cơ chế khác nhau (heap_1 đến heap_5).

2.1.4. Ứng dụng của hệ điều hành FreeRTOS

FreeRTOS được ứng dụng rộng rãi trong nhiều lĩnh vực của hệ thống nhúng nhờ tính linh hoạt và hiệu suất cao. Một số ứng dụng tiêu biểu được trình bày trong bảng sau:

Bảng 2.2: Ứng dụng của FreeRTOS

Lĩnh vực	Ứng dụng cụ thể
IoT	Thiết bị kết nối WiFi, gửi dữ liệu lên cloud (ESP32, AWS IoT)
Công nghiệp	Hệ thống điều khiển tự động, giám sát dây chuyền sản xuất
Robot	Điều khiển chuyển động, xử lý cảm biến đa nhiệm
Y tế	Thiết bị đo lường, giám sát sức khỏe
Điện tử tiêu dùng	Thiết bị âm thanh, cảm biến, hệ thống điều khiển thông minh

2.2. Hệ thống đồng hồ thời gian thực (RTC)

2.2.1. Khái niệm RTC

RTC (Real Time Clock) là một hệ thống hoặc mạch điện tử có chức năng theo dõi và cập nhật thời gian thực, bao gồm giờ, phút, giây, ngày, tháng và năm. RTC hoạt động liên tục ngay cả khi hệ thống chính bị tắt nguồn nhờ sử dụng nguồn pin dự phòng.

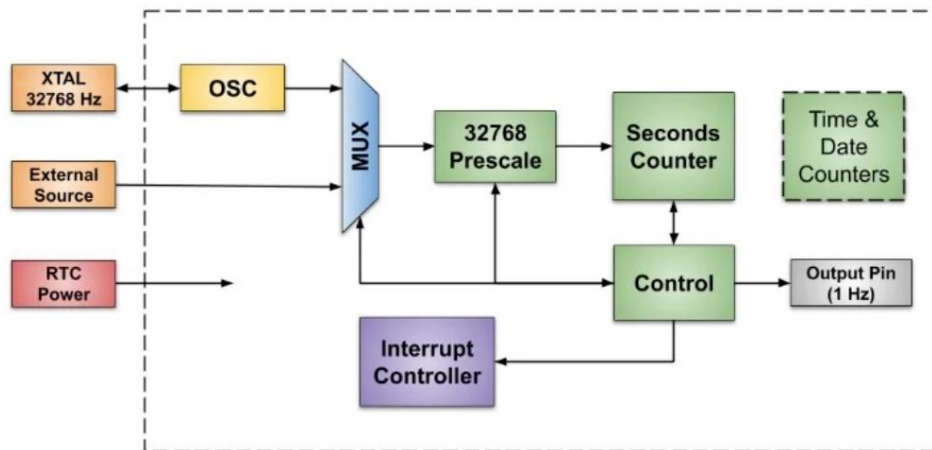
Trong các hệ thống nhúng, RTC đóng vai trò quan trọng trong việc cung cấp thông tin thời gian chính xác để phục vụ cho các ứng dụng như đồng hồ điện tử, hệ thống ghi dữ liệu theo thời gian hoặc các hệ thống điều khiển theo lịch trình.

2.2.2. Nguyên lý hoạt động của RTC

RTC hoạt động dựa trên một bộ dao động thạch anh, thường có tần số 32.768 kHz. Tín hiệu dao động này được chia tần để tạo ra xung nhịp 1 Hz, tương ứng với mỗi giây trôi qua.

Bộ dao động tạo ra xung clock ổn định, sau đó mạch chia tần sẽ giảm tần số xuống còn 1 Hz. Tín hiệu này được đưa vào bộ đếm thời gian để ghi nhận số giây, phút và giờ. Các giá trị thời gian được lưu trữ trong các thanh ghi và liên tục được cập nhật theo thời gian thực.

Nhờ cơ chế này, RTC có thể duy trì độ chính xác cao trong thời gian dài.



Hình 2.1 : Real Time Clock Hardware Features

2.2.3. Phân loại RTC

RTC trong hệ thống nhúng được chia thành hai loại chính là RTC tích hợp trong vi điều khiển và RTC ngoài.

RTC tích hợp trong vi điều khiển là loại được tích hợp sẵn trong các dòng vi điều khiển như STM32 hoặc ESP32. Loại này không cần thêm phần cứng bên ngoài nên giúp tiết kiệm chi phí và không gian thiết kế, tuy nhiên độ chính xác phụ thuộc vào nguồn xung clock của hệ thống.

RTC ngoài là các IC chuyên dụng như DS1307 hoặc DS3231, giao tiếp với vi điều khiển thông qua giao thức I2C. Các module này thường được tích hợp pin dự phòng giúp duy trì thời gian khi mất điện. Đặc biệt, một số IC như DS3231 có khả năng bù nhiệt nên cho độ chính xác cao hơn.

2.2.4. Vai trò của RTC trong đề tài

Trong đề tài thiết kế đồng hồ thời gian thực có LED nhảy theo nhạc sử dụng FreeRTOS, RTC đóng vai trò quan trọng trong việc cung cấp thông tin thời gian chính xác cho toàn bộ hệ thống.

RTC giúp duy trì thời gian liên tục và ổn định, đồng thời cung cấp dữ liệu để hiển thị lên các thiết bị như LED hoặc màn hình. Ngoài ra, RTC còn hỗ trợ việc đồng bộ các tác vụ trong FreeRTOS, ví dụ như cập nhật thời gian theo chu kỳ một giây hoặc điều phối hoạt động giữa các task trong hệ thống.

2.3. Xử lý tín hiệu âm thanh và điều khiển LED RGB theo nhạc

2.3.1. Khái niệm tín hiệu âm thanh số

Trong các hệ thống hiện đại, tín hiệu âm thanh không chỉ tồn tại dưới dạng analog mà còn được số hóa để thuận tiện cho việc xử lý. Quá trình số hóa bao gồm việc lấy mẫu và lượng tử hóa tín hiệu âm thanh, tạo thành chuỗi dữ liệu số theo thời gian.

Tín hiệu âm thanh số cho phép vi điều khiển xử lý linh hoạt hơn, giảm nhiễu và tăng độ chính xác so với tín hiệu analog truyền thống. Các đặc trưng quan trọng vẫn bao gồm biên độ và tần số, trong đó biên độ thường được sử dụng để điều khiển hiệu ứng ánh sáng trong các ứng dụng LED theo nhạc.

2.3.2. Nguyên lý hoạt động của cảm biến âm thanh IMNP441

Cảm biến âm thanh IMNP441 là một microphone MEMS kỹ thuật số, tích hợp bộ chuyển đổi ADC bên trong và xuất dữ liệu thông qua giao tiếp I2S.

Khi có âm thanh tác động, màng MEMS dao động và chuyển đổi thành tín hiệu điện. Tín hiệu này được số hóa ngay trong cảm biến và truyền trực tiếp đến vi điều khiển dưới dạng dữ liệu số thông qua giao thức I2S.

Giao tiếp I2S sử dụng các tín hiệu chính bao gồm xung clock, tín hiệu đồng bộ khung và dữ liệu nối tiếp. Nhờ đó, vi điều khiển có thể nhận được dòng dữ liệu âm thanh liên tục với độ chính xác cao mà không cần sử dụng bộ ADC ngoài.

Việc sử dụng IMNP441 giúp hệ thống giảm nhiễu, tăng độ ổn định và phù hợp với các ứng dụng xử lý tín hiệu số.

2.3.3. Phương pháp xử lý tín hiệu âm thanh

Dữ liệu âm thanh thu được từ cảm biến IMNP441 là chuỗi mẫu số theo thời gian. Để điều khiển LED theo nhạc, hệ thống cần thực hiện các bước xử lý cơ bản.

Trước tiên, các mẫu tín hiệu được thu thập trong một khoảng thời gian ngắn để tạo thành một khung dữ liệu. Sau đó, hệ thống tính toán giá trị biên độ trung bình hoặc giá trị đỉnh của tín hiệu trong khung này.

Giá trị biên độ được sử dụng để xác định cường độ âm thanh tại thời điểm đó. Tùy theo yêu cầu, hệ thống có thể áp dụng thêm các phương pháp lọc nhiễu hoặc chuẩn hóa dữ liệu để đảm bảo tín hiệu ổn định trước khi đưa vào điều khiển LED.

2.3.4. Nguyên lý điều khiển LED WS2812

LED WS2812 là loại LED RGB thông minh, tích hợp IC điều khiển bên trong, cho phép điều khiển từng LED riêng biệt thông qua một chân dữ liệu duy nhất.

Mỗi LED nhận dữ liệu 24 bit bao gồm 8 bit cho mỗi màu đỏ, xanh lá và xanh dương. Dữ liệu được truyền nối tiếp theo chuỗi, LED đầu tiên nhận dữ liệu trước và truyền phần còn lại cho các LED tiếp theo.

Nhờ cơ chế này, hệ thống có thể tạo ra nhiều hiệu ứng ánh sáng khác nhau như đổi màu, chạy đèn hoặc nhấp nháy theo nhạc. Việc điều khiển LED WS2812 yêu cầu tín hiệu thời gian chính xác, do đó thường sử dụng các thư viện hoặc phần cứng hỗ trợ của vi điều khiển.

2.3.5. Điều khiển LED RGB theo nhạc

Dựa trên giá trị biên độ của tín hiệu âm thanh, hệ thống sẽ điều khiển màu sắc và độ sáng của dải LED WS2812.

Khi cường độ âm thanh tăng, hệ thống có thể tăng độ sáng hoặc thay đổi màu sắc của LED theo một quy luật nhất định. Ví dụ, âm thanh nhỏ có thể hiển thị màu xanh, trong khi âm thanh lớn chuyển sang màu đỏ hoặc trắng.

Ngoài ra, với dải LED gồm nhiều phân tử, hệ thống có thể tạo hiệu ứng lan truyền theo nhịp nhạc, giúp tăng tính trực quan và sinh động. Việc sử dụng LED RGB giúp hiển thị phong phú hơn so với LED đơn sắc.

2.3.6. Kết hợp với hệ điều hành FreeRTOS

Trong hệ thống sử dụng FreeRTOS, các chức năng được chia thành các tác vụ độc lập nhằm đảm bảo khả năng xử lý song song.

Một tác vụ đảm nhiệm việc thu thập dữ liệu từ cảm biến IMNP441 thông qua giao tiếp I2S. Một tác vụ khác xử lý dữ liệu âm thanh để tính toán biên độ. Tác vụ điều khiển LED sẽ nhận dữ liệu đã xử lý và cập nhật hiệu ứng ánh sáng cho dải LED WS2812.

Bên cạnh đó, các tác vụ liên quan đến hiển thị thời gian từ RTC vẫn hoạt động đồng thời. Cơ chế lập lịch của FreeRTOS giúp đảm bảo các tác vụ được thực hiện đúng thời điểm, không gây xung đột và duy trì tính thời gian thực của hệ thống.

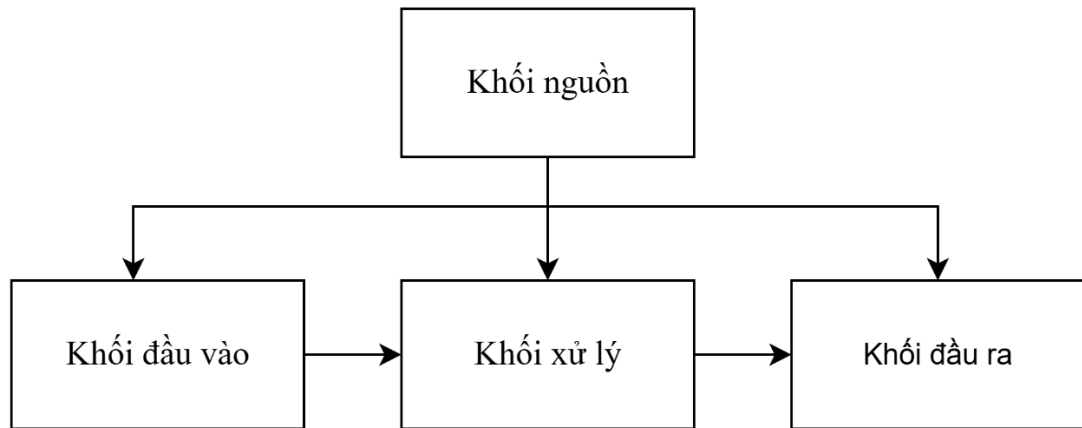
Trong đề tài này, cảm biến IMNP441 được sử dụng để thu nhận tín hiệu âm thanh dưới dạng số với độ chính xác cao. Dữ liệu âm thanh sau khi xử lý sẽ được sử dụng để điều khiển dải LED WS2812, tạo hiệu ứng ánh sáng theo nhạc.

Hệ thống không chỉ hiển thị thời gian thực mà còn phản ứng với âm thanh môi trường, mang lại trải nghiệm trực quan và sinh động. Sự kết hợp giữa xử lý tín hiệu số và điều khiển LED thông minh giúp nâng cao tính hiện đại và hiệu quả của hệ thống.

CHƯƠNG 3 : THIẾT KẾ VÀ THI CÔNG

3.1. Sơ đồ khối của hệ thống

Dựa trên các yêu cầu và mục tiêu đã đề ra, kiến trúc hệ thống được thiết kế bao gồm 4 khối chức năng chính:



Hình 3.1 : Sơ đồ khối

Khối nguồn (Power Supply): Làm nhiệm vụ điều áp và cung cấp năng lượng ổn định cho toàn bộ hệ thống. Do vi điều khiển STM32F4 và các IC ngoại vi hoạt động ở mức logic 3.3V, trong khi hệ thống LED thường yêu cầu nguồn 5V, khối nguồn được thiết kế để cung cấp song song và an toàn cả hai mức điện áp này.

Khối xử lý trung tâm: Đóng vai trò là "bộ não" của hệ thống, sử dụng vi điều khiển STM32F4. Khối này đảm nhiệm việc thu thập dữ liệu chuẩn xác từ khối RTC, đọc và phân tích tín hiệu từ cảm biến âm thanh qua bộ chuyển đổi ADC, tiếp nhận các lệnh từ giao diện người dùng và xuất tín hiệu điều khiển ra khối đầu ra.

Khối đầu vào: Có nhiệm vụ cung cấp dữ liệu cho hệ thống, bao gồm thông tin thời gian thực và tín hiệu âm thanh từ môi trường. Trong đó, module RTC DS1307 đảm nhận việc cung cấp dữ liệu thời gian chính xác thông qua giao tiếp I2C và có khả năng duy trì hoạt động nhờ pin dự phòng khi mất điện. Bên cạnh đó, cảm biến âm thanh số IMNP441 sử dụng giao tiếp I2S để truyền dữ liệu âm thanh đã được số hóa trực tiếp đến vi điều khiển, giúp giảm nhiễu và nâng cao độ

ổn định. Hai nguồn dữ liệu này là cơ sở để khối xử lý trung tâm thực hiện các thuật toán xử lý và điều khiển hệ thống. Ngoài ra còn sử dụng 1 số cảm biến chạm để xử lý 1 số chức năng đặc biệt như hẹn báo thức.

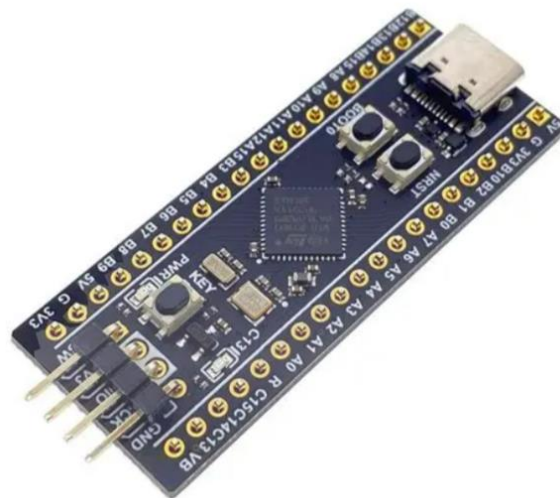
Khối đầu ra: Có nhiệm vụ hiển thị kết quả xử lý của hệ thống ra bên ngoài thông qua màn hình OLED SSD1306 và dải LED RGB WS2812. Màn hình OLED sử dụng giao tiếp I2C để hiển thị thời gian thực với độ rõ nét cao và tiêu thụ năng lượng thấp. Trong khi đó, dải LED WS2812 cho phép điều khiển từng LED riêng biệt thông qua một đường dữ liệu duy nhất, từ đó tạo ra các hiệu ứng ánh sáng theo nhạc dựa trên tín hiệu âm thanh đã được xử lý. Sự kết hợp giữa hiển thị thông tin và hiệu ứng ánh sáng giúp hệ thống trở nên trực quan và sinh động hơn.

3.2. Thi công phần cứng

3.2.1. Các linh kiện sử dụng

3.2.1.1. STM32F411CEU6

STM32F411CEU6 là vi điều khiển thuộc dòng STM32F4 của hãng STMicroelectronics, được sử dụng làm bộ xử lý trung tâm trong hệ thống. Đây là dòng vi điều khiển hiệu năng cao, tích hợp lõi ARM Cortex-M4 với khả năng xử lý số và tín hiệu mạnh mẽ, rất phù hợp cho các ứng dụng thời gian thực và xử lý tín hiệu như đề tài đồng hồ kết hợp LED theo nhạc.



Hình 3.2 : STM32F411CEU6

Dưới đây là bảng thông số kỹ thuật:

Bảng 3.1 : Thông số kỹ thuật của STM32F411CEU6

Thông số	Giá trị
Lõi xử lý	ARM Cortex-M4
Tần số hoạt động tối đa	100 MHz
Bộ nhớ Flash	512 KB
SRAM	128 KB
Điện áp hoạt động	1.7V – 3.6V
Giao tiếp	I2C, SPI, I2S, USART, USB
ADC	12-bit
Timer	Nhiều timer 16-bit và 32-bit
PWM	Có hỗ trợ
Số chân I/O	~20
Giao tiếp âm thanh	I2S
Debug	SWD

Với bộ nhớ Flash 512 KB và SRAM 128 KB, đủ lớn để triển khai hệ điều hành FreeRTOS cùng nhiều tác vụ song song như đọc dữ liệu từ cảm biến âm thanh, xử lý tín hiệu, điều khiển LED và hiển thị thời gian. Ngoài ra, STM32F411CEU6 hỗ trợ đầy đủ các giao tiếp ngoại vi cần thiết cho đề tài. Giao tiếp I2C được sử dụng để kết nối với module RTC DS1307 và màn hình OLED SSD1306, trong khi giao tiếp I2S được sử dụng để thu nhận dữ liệu âm thanh số từ cảm biến IMNP441. Điều này giúp hệ thống tận dụng tối đa khả năng xử lý tín hiệu số mà không cần thêm phần cứng trung gian.

Bên cạnh đó, vi điều khiển còn có khả năng tạo tín hiệu PWM và điều khiển chính xác timing, rất phù hợp để điều khiển dải LED WS2812 yêu cầu xung thời

gian nghiêm ngặt. Tốc độ xử lý cao lên đến 100 MHz giúp đảm bảo hệ thống hoạt động mượt mà ngay cả khi chạy đa nhiệm với FreeRTOS.

STM32F411CEU6 có thể được lập trình bằng ngôn ngữ C/C++ thông qua các môi trường phát triển như STM32CubeIDE, Keil uVision hoặc PlatformIO. Hệ sinh thái thư viện HAL và CMSIS của ST giúp việc phát triển phần mềm trở nên thuận tiện, đồng thời hỗ trợ tốt cho việc tích hợp FreeRTOS vào hệ thống.

3.2.1.2. RTC DS1307

DS1307 là một mạch tích hợp đồng hồ thời gian thực (RTC) được sử dụng để cung cấp thông tin thời gian chính xác cho hệ thống, bao gồm giây, phút, giờ, ngày, tháng và năm. Trong đề tài này, DS1307 đóng vai trò là nguồn dữ liệu thời gian chính, giúp hệ thống duy trì và cập nhật thời gian liên tục.



Hình 3.3 : Module DS1307

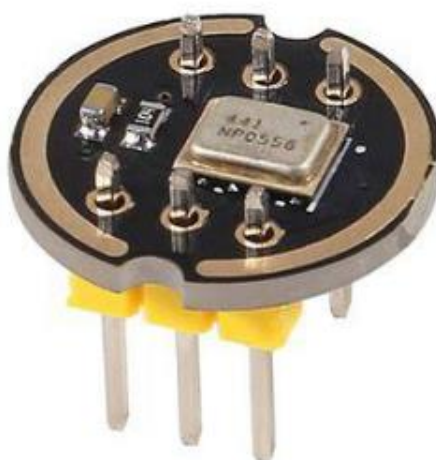
Module DS1307 sử dụng giao tiếp I2C để truyền dữ liệu đến vi điều khiển STM32F411CEU6, giúp tiết kiệm chân kết nối và dễ dàng tích hợp với các thiết bị ngoại vi khác như màn hình OLED SSD1306. Một ưu điểm quan trọng của DS1307 là khả năng hoạt động với pin dự phòng (thường là pin CR2032), cho phép duy trì thời gian ngay cả khi hệ thống chính bị mất nguồn.

Bảng 3.2 : Thông số của DS1307:

Thông số	Giá trị
Điện áp hoạt động	4.5V – 5.5V
Giao tiếp	I2C
Tần số thạch anh	32.768 kHz
RAM tích hợp	56 byte
Chế độ giờ	12h / 24h

3.2.1.3. IMNP441

INMP441 là microphone MEMS kỹ thuật số sử dụng giao tiếp I2S, được dùng để thu nhận tín hiệu âm thanh trong hệ thống. Cảm biến này tích hợp sẵn bộ ADC bên trong, cho phép chuyển đổi trực tiếp tín hiệu âm thanh thành dữ liệu số và truyền đến vi điều khiển mà không cần mạch chuyển đổi ngoài.



Hình 3.4 : IMNP441

INMP441 hoạt động bằng cách lấy mẫu tín hiệu âm thanh và xuất dữ liệu số theo chuẩn I2S với độ phân giải cao, giúp giảm nhiễu và tăng độ chính xác so với cảm biến analog. Trong đề tài, STM32F411CEU6 sử dụng giao tiếp I2S để đọc

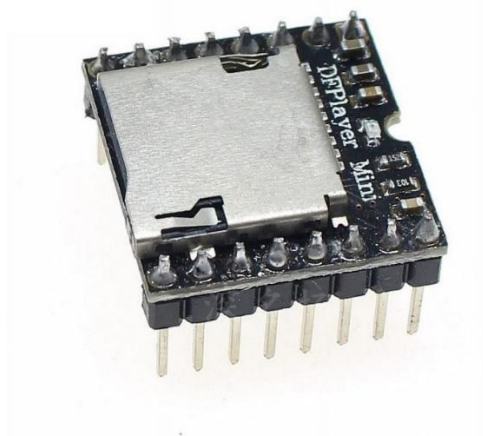
dữ liệu từ cảm biến, từ đó tính toán biên độ âm thanh phục vụ điều khiển LED RGB theo nhạc.

Bảng 3.3 : Thông số kỹ thuật IMNP441

Thông số	Giá trị
Loại	Microphone MEMS số
Giao tiếp	I2S
Điện áp hoạt động	1.8V – 3.3V
Độ phân giải	24-bit
Dải tần	~60 Hz – 15 kHz
SNR	~61 dB

3.2.1.4. DFPLAYER

DFPlayer Mini là module giải mã âm thanh MP3 nhỏ gọn, tích hợp sẵn bộ giải mã cứng cho các định dạng MP3, WAV, WMA và bộ khuếch đại công suất bên trong. Module hỗ trợ đọc dữ liệu trực tiếp từ thẻ nhớ microSD và giao tiếp với vi điều khiển STM32F4 thông qua giao thức UART. Trong đề tài, DFPlayer được sử dụng để phát các tệp âm thanh hoặc âm báo chất lượng cao, giúp tối ưu hóa tài nguyên xử lý của vi điều khiển trung tâm nhờ bộ chuyển đổi DAC 24-bit tích hợp sẵn.

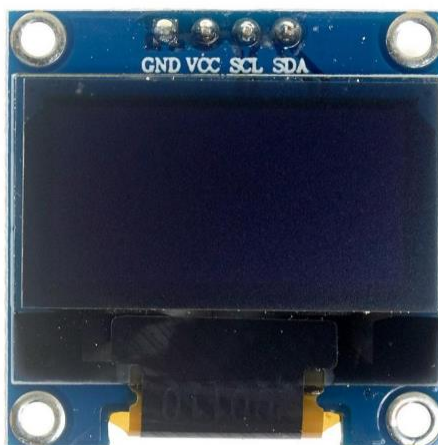


Hình 3.5: DFPLAYER

3.2.1.5. Oled SSD 1306

SSD1306 là màn hình OLED kích thước nhỏ, thường có độ phân giải 128x64, được sử dụng để hiển thị thông tin trong hệ thống. Màn hình sử dụng giao tiếp I2C giúp tiết kiệm chân kết nối và dễ dàng tích hợp với vi điều khiển STM32F411CEU6.

OLED có ưu điểm hiển thị rõ nét, độ tương phản cao và tiêu thụ năng lượng thấp do không cần đèn nền. Trong đề tài, SSD1306 được dùng để hiển thị thời gian thực từ RTC và các thông tin trạng thái của hệ thống, giúp tăng tính trực quan và dễ quan sát.



Hình 3.6 : OLED SSD 1306

Bảng 3.4: Thông số của OLED SSD1306

Thông số	Giá trị
Loại màn hình	OLED
Độ phân giải	128 x 64 pixel
Kích thước	0.96 inch
Điện áp hoạt động	3.3V – 5V
Giao tiếp	I2C / SPI
Công suất tiêu thụ	Thấp

3.2.1.6. Dải LED WS2812

WS2812 là LED RGB thông minh tích hợp IC điều khiển bên trong, cho phép điều khiển từng LED riêng biệt thông qua một chân dữ liệu duy nhất. Mỗi LED có thể hiển thị nhiều màu sắc khác nhau bằng cách điều chỉnh giá trị 8 bit cho ba kênh màu đỏ, xanh lá và xanh dương.

Trong đề tài, dải LED WS2812 được sử dụng để hiển thị hiệu ứng ánh sáng theo nhạc dựa trên tín hiệu âm thanh từ cảm biến INMP441. Nhờ khả năng điều khiển linh hoạt và đồng bộ cao, LED có thể tạo ra các hiệu ứng như nhấp nháy, đổi màu hoặc lan truyền theo biên độ âm thanh.

Bảng 3.5: Thông số của LED WS2812

Thông số	Giá trị
Loại LED	RGB tích hợp IC
Điện áp hoạt động	5V
Giao tiếp	1-wire (Data)
Độ phân giải màu	24-bit (8 bit mỗi màu)
Tần số truyền	~800 kHz



Hình 3.7 : LED WS2812

3.2.1.7. Nút nhấn switch

Nút nhấn được sử dụng trong hệ thống là loại công tắc cơ học (Mechanical Switch). Thiết bị đóng vai trò nhận tín hiệu điều khiển từ người dùng thông qua việc đóng/ngắt các tiếp điểm kim loại bên trong khi có lực tác động lên phần trục. Switch được kết nối trực tiếp với các chân GPIO của vi điều khiển để thực hiện

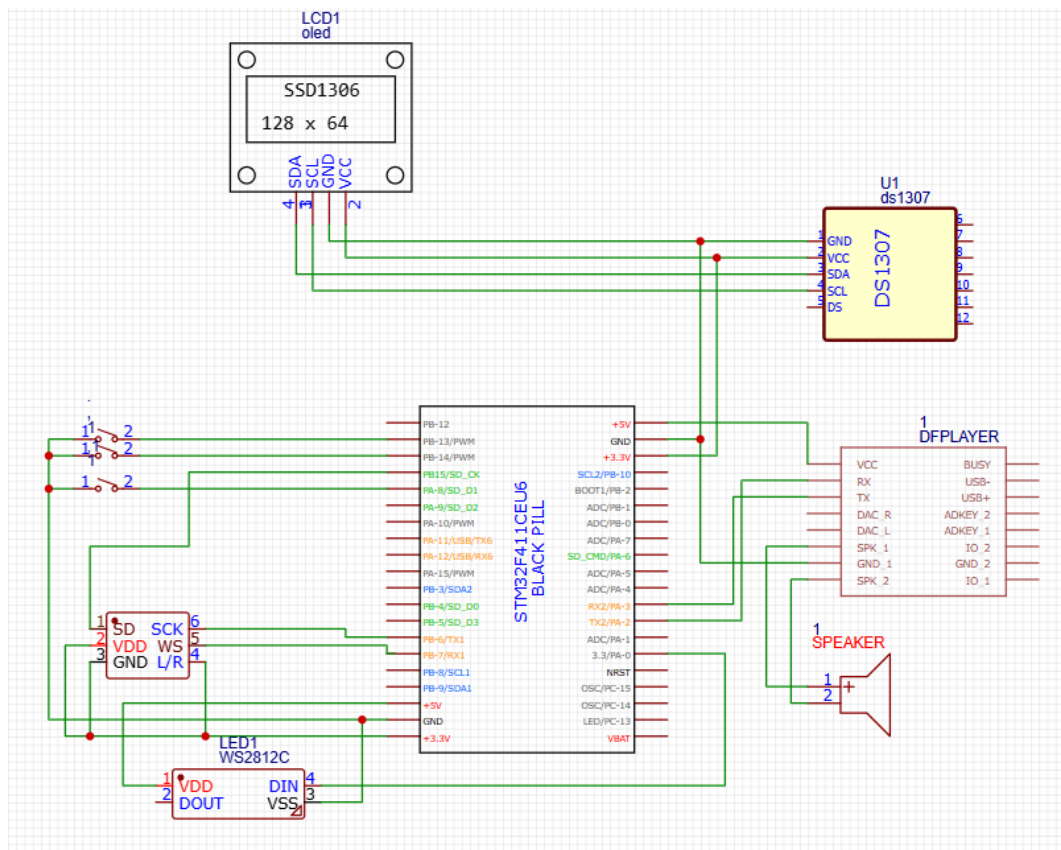
các chức năng như cấu hình thời gian thực và chuyển đổi các hiệu ứng nháy LED. Đặc điểm của loại này là độ bền cao, phản hồi lực tốt và tín hiệu đầu ra ổn định sau khi được xử lý chống rung bằng phần mềm.



Hình 3.8 : Nút nhấn switch

3.2.2. Sơ đồ kết nối:

Hình ảnh dưới đây là sơ đồ kết nối của toàn bộ hệ thống:

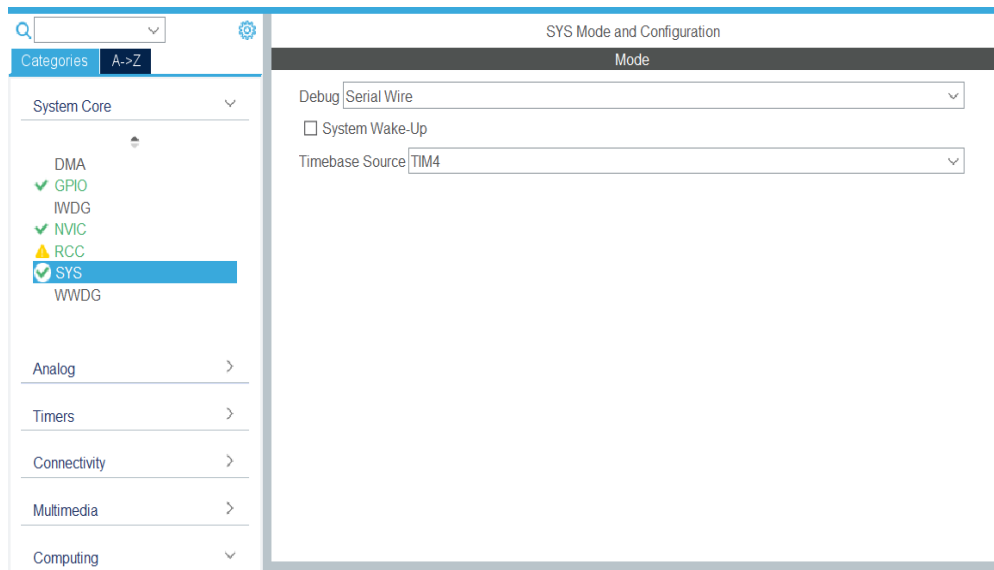


Hình 3.9 : Sơ đồ kết nối của hệ thống

3.2. Thi công phần mềm

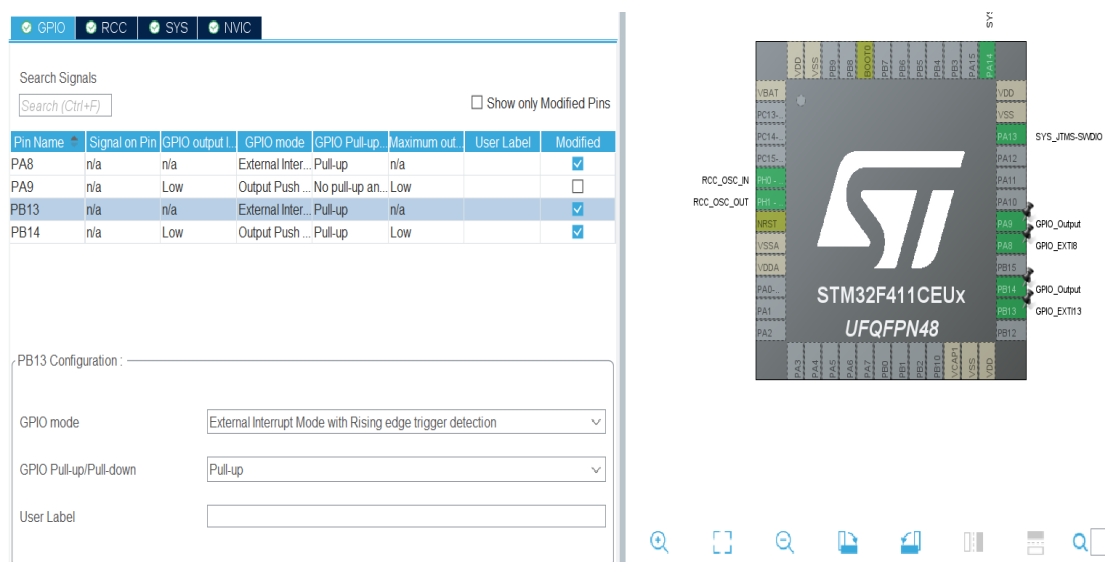
3.2.1. Cấu hình trên STM32 CubeMX

Bước 1: Cấu hình thạch anh là *Crystal/Ceramic Resonator* và Debug chọn *Serial Wire*, khi dùng RTOS ta chọn TIM4:



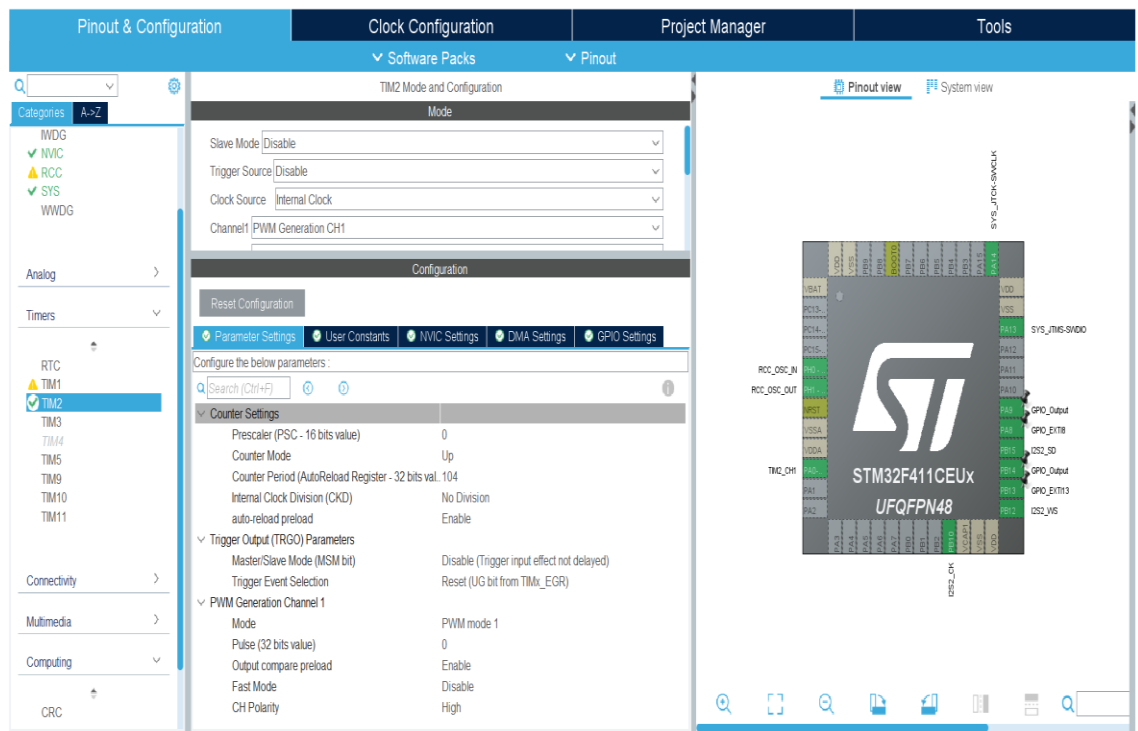
Hình 3.10 :Cấu hình thạch anh và Debug

Bước 2: Cấu hình GPIO cho các chân cảm biến chạm và còi báo, GPIO mode External Interrupt Mode with Rising edge cho GPIO PA8 và PB13



Hình 3.11 : Cấu hình GPIO

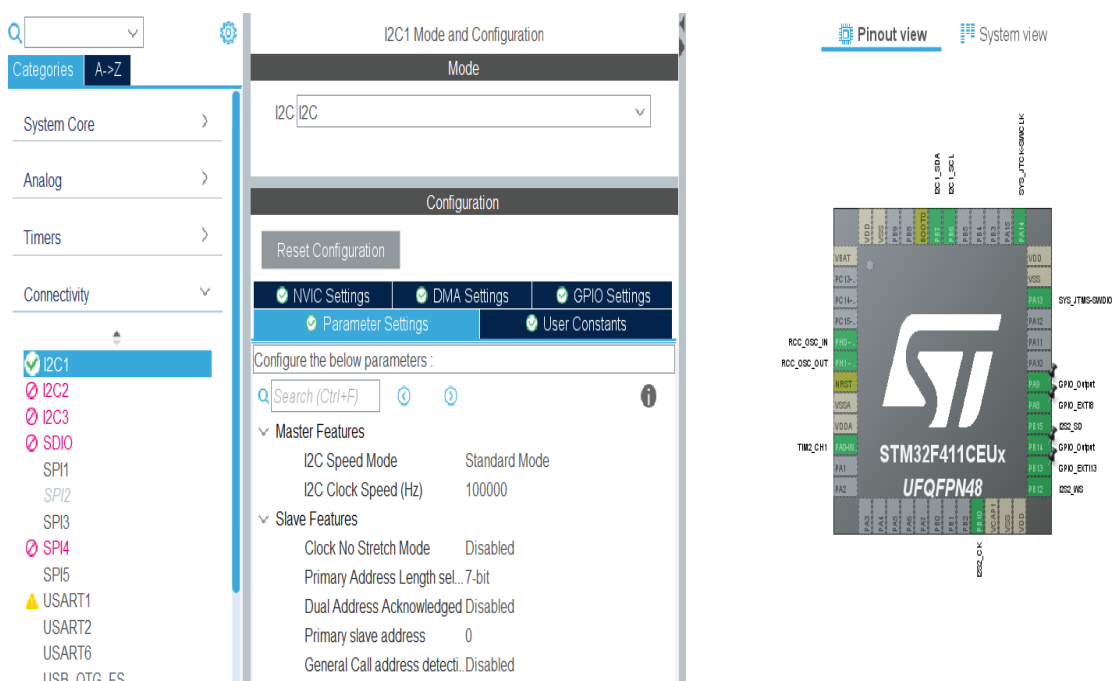
Bước 3: Cấu hình I2S để đọc cảm biến âm thanh, mode Master Receive và DMA Setting :



Hình 3.12 : Cấu hình I2S

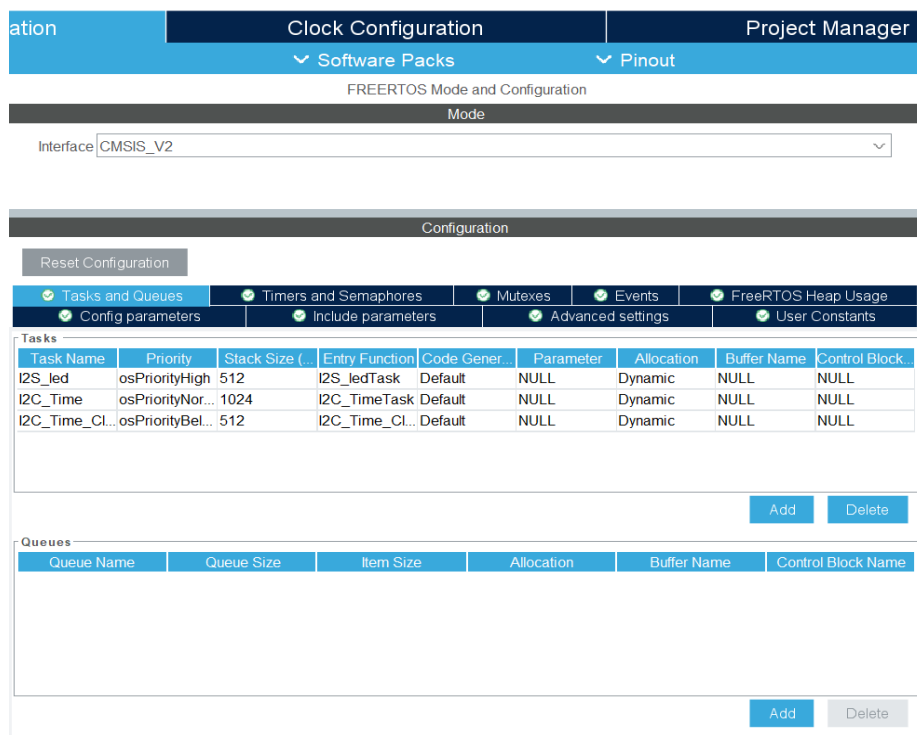
Bước 4: Cấu hình timer điều khiển LED: Chọn TIM2, Clock Source là *Internal Clock*, Channel 1 là *PWM Generation CH1* và Period (ARR) là 104

Bước 5: Cấu hình I2C (Cho OLED và RTC):



Hình 3.13 : Cấu hình I2C

Bước 6: Cấu hình Middleware: FreeRTOS, đây là bước quan trọng nhất để hệ thống chạy được FREERTOS, interface chọn CMSIS_V2, sau đó tạo 3 task chính như hình dưới đây:



Hình 3.14 : Cấu hình FreeRTOS

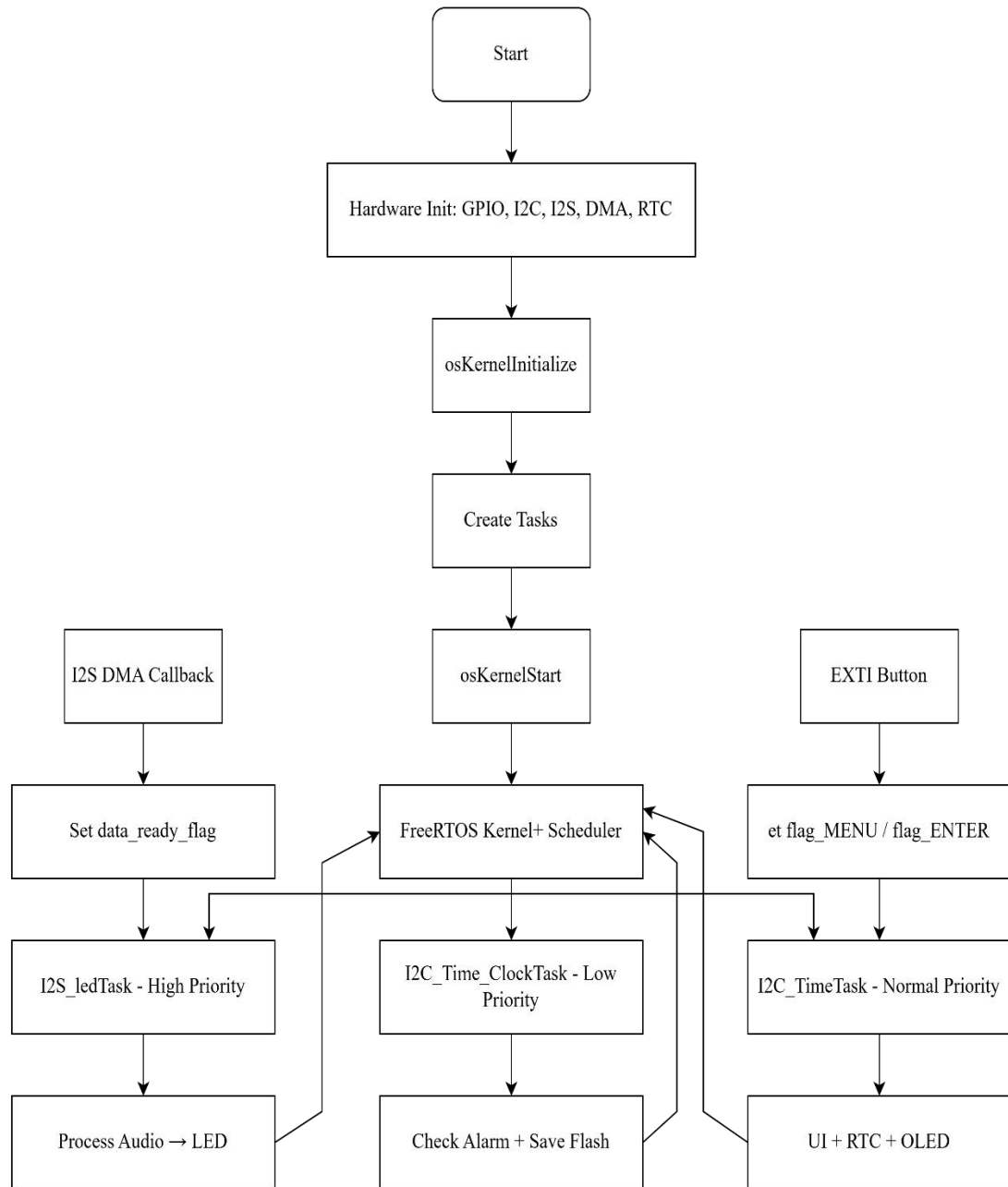
Bước 7: Cấu hình Xung nhịp (Clock Configuration) cho hệ thống chạy với HCLK là 84 MHz.

3.2.2. Triển khai chương trình

Chương trình được viết và triển khai trên phần mềm STM32CubeIDE.

Chương trình bắt đầu bằng việc khởi tạo các phần cứng như GPIO, I2C, I2S, DMA và RTC. Sau đó hệ thống gọi hàm osKernelInitialize, tạo các task cần thiết rồi khởi động nhân FreeRTOS bằng osKernelStart. Khi nhân hệ điều hành chạy, bộ lập lịch sẽ quản lý ba task với các mức ưu tiên khác nhau. Task I2S_ledTask có ưu tiên cao nhất và chỉ hoạt động khi ngắt DMA của I2S đặt cờ data_ready_flag. Task này nhận dữ liệu âm thanh từ I2S rồi xử lý để điều khiển LED hiển thị theo tín hiệu âm thanh. Task I2C_Time_ClockTask có ưu tiên thấp,

dùng để kiểm tra báo thức và lưu dữ liệu xuống Flash. Ngoài ra, khi người dùng nhấn nút ngoài qua ngắt EXTI, các cờ MENU và ENTER sẽ được thiết lập để kích hoạt task giao diện. Task I2C_TimeTask ở mức ưu tiên trung bình sẽ cập nhật giao diện, đọc thời gian từ RTC và hiển thị lên OLED.

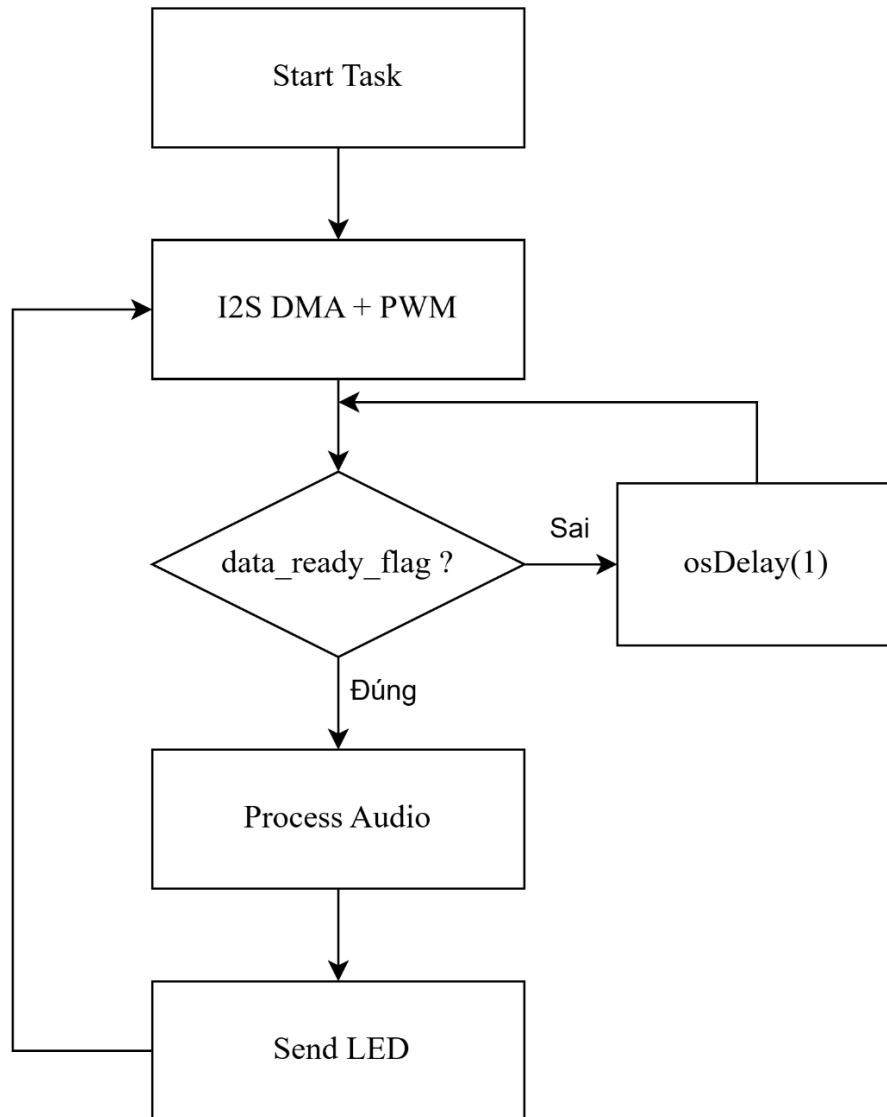


Hình 3.15 : Lưu đồ giải thuật tổng thể của hệ thống

Phần chương trình được trình bày chi tiết tại phần **PHỤ LỤC**.

3.2.2.1. I2S_ledTask (High Priority)

Task bắt đầu bằng cách khởi tạo DMA cho I2S và PWM để chuẩn bị nhận dữ liệu âm thanh. Sau đó chương trình liên tục kiểm tra cờ `data_ready_flag`; nếu chưa có dữ liệu thì task tạm dừng bằng `osDelay(1)`. Khi cờ được đặt, task sẽ xử lý tín hiệu âm thanh nhận được. Cuối cùng, dữ liệu sau xử lý sẽ được gửi ra LED rồi quay lại vòng lặp để tiếp tục cập nhật.

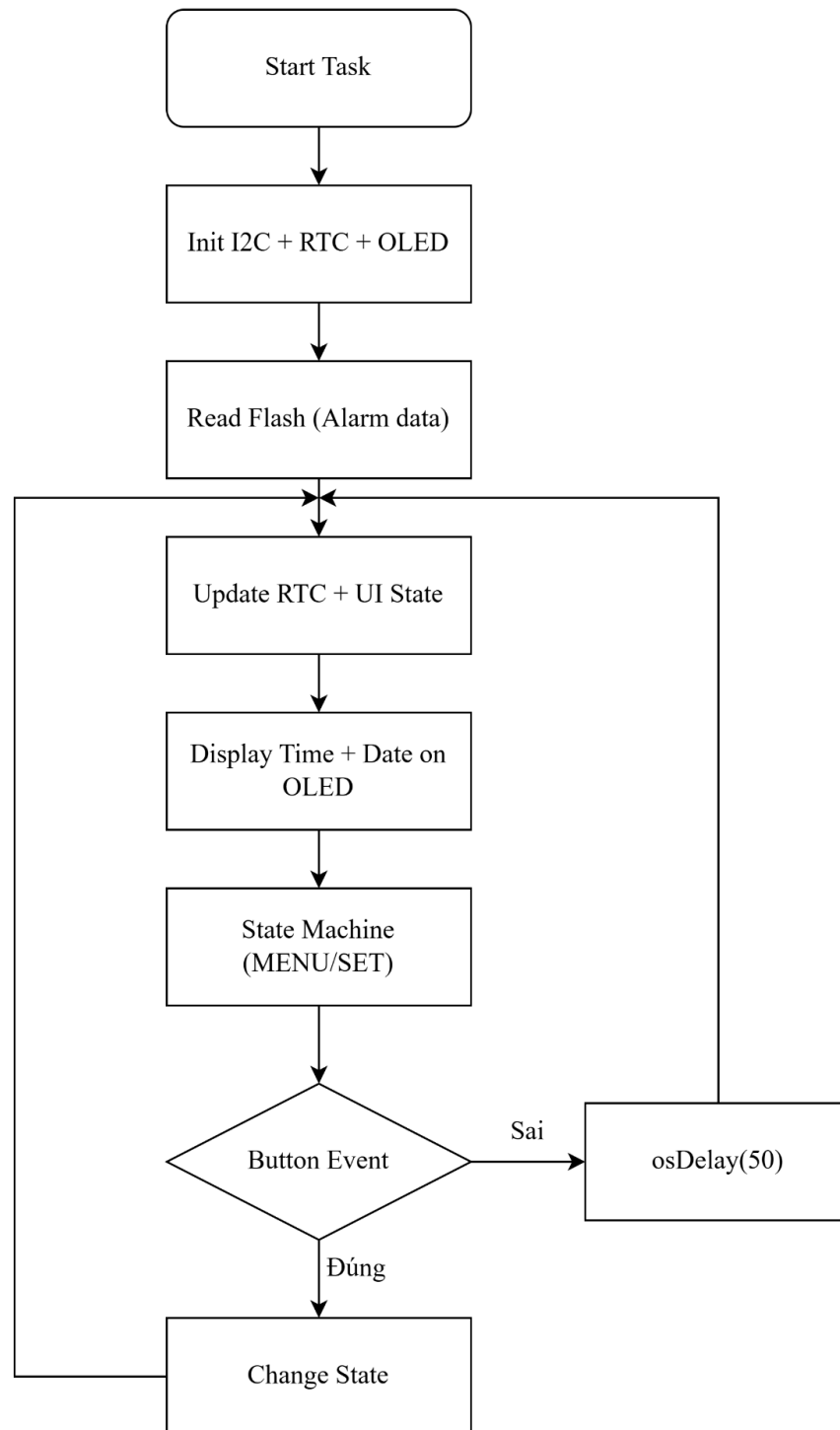


Hình 3.16 : Thuật toán mô tả chương trình task `I2S_ledTASK`

3.2.2.2. I2C_TimeTask (UI + OLED)

Task bắt đầu bằng việc khởi tạo giao tiếp I2C, RTC và màn hình OLED. Sau đó chương trình đọc dữ liệu báo thức đã lưu trong Flash để khôi phục trạng thái trước đó. Trong vòng lặp chính, task liên tục cập nhật thời gian từ RTC, cập nhật

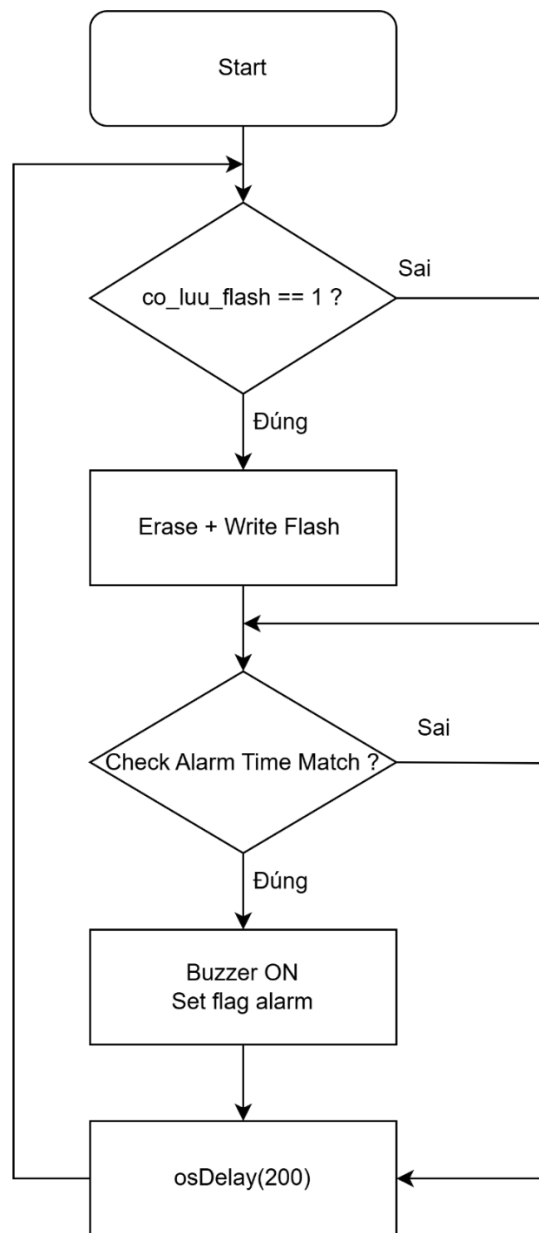
trạng thái giao diện và hiển thị ngày giờ lên OLED. Chương trình sử dụng state machine để xử lý các chế độ MENU và SET. Nếu có sự kiện nhấn nút thì trạng thái sẽ được thay đổi, còn nếu không task sẽ chờ `osDelay(50)` rồi tiếp tục lặp.



Hình 3.17 : Thuật toán mô tả chương trình task `I2C_TimeTask`

3.2.2.3. I2C_Time_ClockTask

Task này dùng để kiểm tra trạng thái báo thức theo thời gian lấy từ RTC. Nếu thời gian hiện tại trùng với thời gian báo thức đã lưu, chương trình sẽ kích hoạt còi báo thức hoặc bật thiết bị tương ứng. Khi người dùng thay đổi thông tin báo thức, dữ liệu mới sẽ được ghi lại vào Flash để không bị mất sau khi tắt nguồn. Sau mỗi lần kiểm tra, task sẽ chờ một khoảng thời gian ngắn rồi tiếp tục lặp.



Hình 3.18 : Thuật toán mô tả chương trình task I2C_Time_ClockTask

CHƯƠNG 4 : KẾT QUẢ VÀ ĐÁNH GIÁ

4.1. Sản phẩm thực tế



Hình 4.1 : Sản phẩm thực tế



Hình 4.2: Sản phẩm thực tế 2

4.2 Kiểm tra và đánh giá

Sau khi hoàn thiện thi công phần cứng và nạp chương trình, nhóm đã tiến hành các thử nghiệm thực tế để kiểm tra tính ổn định và độ chính xác của hệ thống. Quá trình đánh giá tập trung vào khả năng vận hành đa nhiệm của FreeRTOS và sự phối hợp giữa các khối chức năng.

4.2.1. Kết quả thực hiện các chức năng chính

Hệ thống đã trải qua các bài kiểm tra chức năng với kết quả được ghi nhận trong bảng sau:

Bảng 4.1: Kết quả thử nghiệm các tính năng của hệ thống

STT	Tính năng kiểm tra	Nội dung thực hiện	Kết quả thực hiện
1	Thiết lập thời gian	Thay đổi Ngày/Tháng/Năm, Giờ/Phút thông qua giao diện điều khiển.	Cập nhật chính xác dữ liệu vào thanh ghi của RTC DS1307 qua giao tiếp I2C.
2	Độ chính xác thời gian	So sánh giờ hiển thị trên OLED với đồng hồ tiêu chuẩn trong vòng 24 giờ.	Thời gian chạy ổn định, đồng nhất với thực tế nhờ cơ chế đếm xung thạch anh 32.768 kHz.
3	Chức năng Báo thức	Đặt giờ báo thức và kiểm tra phản hồi của hệ thống.	Khi thời gian thực trùng với giờ hẹn, hệ thống kích hoạt nhạc báo thức đã được cài đặt
4	Quản lý âm nhạc	Lựa chọn và phát các tệp tin âm thanh lưu trữ trong thẻ nhớ microSD.	Module DFPlayer truy xuất và giải mã đúng tệp MP3 đã chọn qua giao tiếp UART, âm thanh rõ ràng.
5	Lưu trữ dữ liệu	Tắt nguồn đột ngột và kiểm tra lại thông tin báo thức đã cài đặt.	Thông tin báo thức được khôi phục chính xác nhờ dữ liệu đã lưu vào Flash trước đó.

4.2.2. Đánh giá hiệu năng hệ thống

Tính ổn định của hệ điều hành: Nhờ cấu trúc đa nhiệm của FreeRTOS, việc phát nhạc và cập nhật thời gian diễn ra song song mà không xuất hiện tình trạng

trễ hay đứng máy (freeze). Task ưu tiên cao (I2S_ledTask) xử lý tín hiệu âm thanh cực nhanh, giúp LED nháy bám sát nhịp nhạc.

Giao diện người dùng (UI): Màn hình OLED SSD1306 hiển thị thông tin trực quan, các chế độ MENU và SET hoạt động mượt mà nhờ thuật toán State Machine.

Khả năng lưu trữ: Việc kết hợp giữa RTC DS1307 (duy trì thời gian thực bằng pin dự phòng) và Flash (lưu cấu hình báo thức) giúp hệ thống hoạt động tin cậy trong điều kiện nguồn điện không ổn định.

KẾT LUẬN

Sau quá trình nghiên cứu, thiết kế và thi công hệ thống “Đồng hồ thời gian thực kết hợp LED hiệu ứng âm thanh sử dụng FreeRTOS”, nhóm đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu. Sản phẩm không chỉ là một thiết bị đo lường thời gian đơn thuần mà còn là sự kết hợp tinh tế giữa kỹ thuật xử lý tín hiệu số và nghệ thuật ánh sáng.

1. Ưu điểm

Hiệu năng xử lý vượt trội: Việc chuyển đổi từ kiến trúc 8-bit sang vi điều khiển 32-bit ARM Cortex-M4 (STM32F411) giúp hệ thống có khả năng tính toán mạnh mẽ, đáp ứng tốt các thuật toán xử lý tín hiệu theo thời gian thực (Real-time).

Quản lý đa nhiệm ổn định: Nhờ ứng dụng hệ điều hành FreeRTOS, các tác vụ như cập nhật thời gian từ RTC, hiển thị OLED và xử lý âm thanh được phân tách độc lập, đảm bảo tính đồng bộ và không gây xung đột tài nguyên.

Độ chính xác và tính ổn định cao:

Sử dụng cảm biến âm thanh số IMNP441 qua giao tiếp I2S giúp loại bỏ nhiễu analog, mang lại dữ liệu âm thanh trung thực.

Module RTC DS1307 đảm bảo duy trì thời gian liên tục ngay cả khi mất nguồn nhờ pin dự phòng.

Hiệu ứng ánh sáng sinh động: Dải LED WS2812 kết hợp với thuật toán phân tích biên độ âm thanh tạo ra các hiệu ứng chuyển màu mượt mà, bám sát nhịp điệu (Audio-reactive).

Giao diện trực quan: Màn hình OLED SSD1306 cung cấp khả năng hiển thị rõ nét với độ tương phản cao, tối ưu hóa trải nghiệm người dùng.

2. Nhược điểm

Độ chính xác thời gian dài hạn: Mặc dù RTC DS1307 hoạt động ổn định, nhưng so với các dòng IC chuyên dụng có bù nhiệt như DS3231, sai số về thời gian có thể tích lũy theo năm tháng do ảnh hưởng của nhiệt độ môi trường.

Thuật toán xử lý âm thanh: Hiện tại, hệ thống chủ yếu xử lý dựa trên biên độ (Volume-based). Việc chưa áp dụng triệt để các thuật toán phân tích phổ sâu như FFT (Fast Fourier Transform) khiến hiệu ứng LED chưa thể phân tách chi tiết theo dải tần số (Bass, Mid, Treble).

Khả năng tương tác: Hệ thống sử dụng cảm biến chạm cơ bản, chưa có ứng dụng di động hoặc giao diện điều khiển từ xa để tùy biến các chế độ LED phức tạp.

3. Phương hướng phát triển

Nâng cao thuật toán DSP: Triển khai thuật toán biến đổi Fourier nhanh (FFT) để phân tích phổ âm thanh chuyên sâu, tạo ra các hiệu ứng nháy LED theo tần số nhạc chuyên nghiệp hơn.

Tích hợp kết nối IoT: Ứng dụng các tính năng hỗ trợ Internet of Things của FreeRTOS để kết nối hệ thống với Cloud (như AWS), cho phép đồng bộ thời gian thực qua NTP (Network Time Protocol) và điều khiển qua Smartphone.

Cải tiến phần cứng: Thay thế DS1307 bằng DS3231 để đạt độ chính xác cực cao về thời gian.

Tích hợp thêm module giải mã âm thanh để thiết bị có thể phát nhạc trực tiếp thay vì chỉ nhận tín hiệu từ môi trường.

Tối ưu hóa UI/UX: Thiết kế vỏ hộp bằng công nghệ in 3D kết hợp với các vật liệu tán xạ ánh sáng để tăng tính thẩm mỹ cho sản phẩm thương mại.

TÀI LIỆU THAM KHẢO

- [1] FreeRTOS Official Documentation, "FreeRTOS Kernel Guide and Reference Manual", truy cập tại: freertos.org.
- [2] STMicroelectronics, "STM32F411xC/E Advanced Arm®-based 32-bit MCUs Datasheet", Tài liệu kỹ thuật dòng vi điều khiển STM32F4.
- [3] Trần Thu Hà, Đỗ Mạnh Hà, Xử lý tín hiệu số, Nhà xuất bản Giáo dục Việt Nam (Tài liệu lưu hành nội bộ ĐH Công nghiệp Hà Nội), 2020.
- [4] Maxim Integrated, "DS1307 Serial, I2C Real-Time Clock Datasheet", Tài liệu kỹ thuật IC đồng hồ thời gian thực.
- [5] InvenSense (TDK), "INMP441 Omnidirectional Microphone with I2S Digital Output Datasheet", Tài liệu kỹ thuật cảm biến âm thanh MEMS.
- [6] Nguyễn Hữu Thanh, Giáo trình Hệ điều hành, Nhà xuất bản Bách Khoa Hà Nội, 2021.